Simulator Scene Configuration Guide

Robotics and Cyber-Physical Systems
School of Sciences and Engineering
Tecnológico de Monterrey

This guide explains how to configure the Stretch simulator's scene. These settings only affect simulation; the same robot-control scripts continue to work on the physical robot.

1 Getting Started

Update the repository before running a scenario:

```
git pull
git submodule update --init --recursive
uv sync
```

All scene settings live in the root-level `sim_config.py`. Select an included scenario with its final line, then restart the simulated robot:

```
CONFIG = _CUBES
```

Start by trying the included scenarios and exploring each scene with normal teleop controls.

2 Included Scenarios

Configuration	What it shows
`_DEFAULT`	Default table scene and robot pose.
`_OBJECT`	One imported textured Android Lego mesh.
`_CUBES`	Dynamic, kinematic, fixed, colliding, and ghost cubes.
`_TEXTURED_CUBE`	A six-face texture atlas on a kinematic cube.
`_ARUCO_CUBES`	One-face and six-face ArUco cubes.
`_ANDROID_ARMY`	Many independently movable imported meshes.
`_ROBOT_POSE`	A changed robot starting pose.
`_ROBOCASA`	A generated RoboCasa kitchen.
`_ROBOCASA_OBJECT`	An imported mesh inside a RoboCasa kitchen.

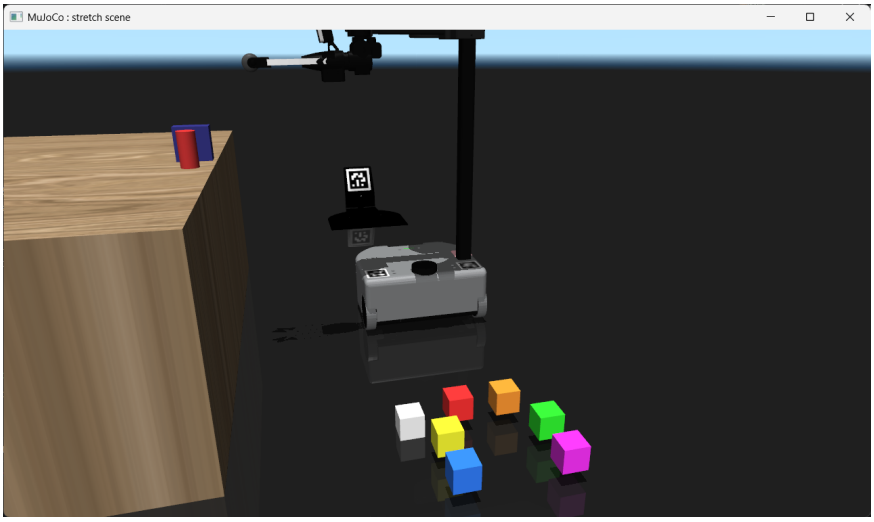

Figure 1: The `_CUBES` configuration after launch. The gravity-enabled white cube has fallen; the remaining cubes stay suspended.

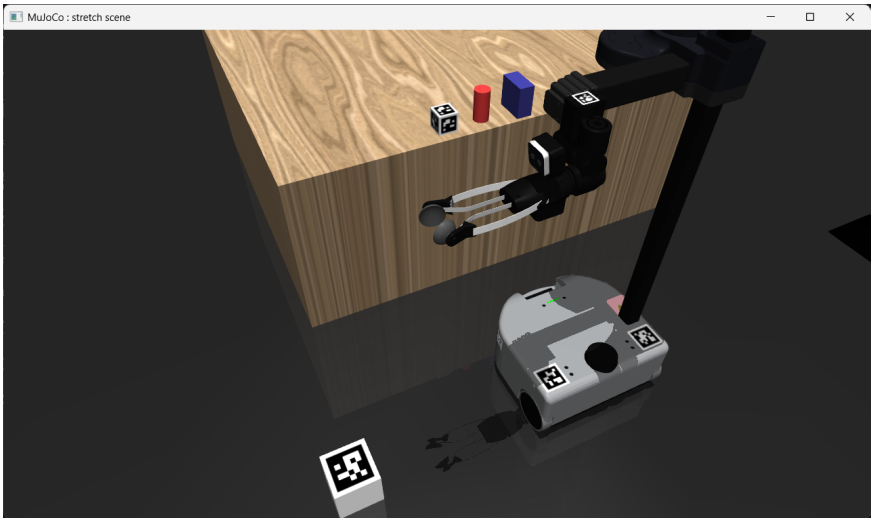

Figure 2: The `_ARUCO_CUBES` configuration, including the six-face marker cube on the table.

3 The `SimConfig` Object

Every scenario is a `SimConfig`. Its most useful fields are:

- `objects`: meshes, cubes, and ArUco cubes to add to the scene.
- `robot_pose`: optional initial position and quaternion for the robot.
- `robocasa`: enables a generated kitchen and selects its task, layout, and style.
- `scene`: optional custom MuJoCo scene file. Leave it unset to use the default table scene.
- `headless`, `show_viewer_ui`, `camera_rate`, and `timestep`: viewer, camera, and simulation settings.

4 Objects and Physics

`MeshObject.from_folder()` imports an OBJ mesh and, when present, its `texture.png`. `Cube` creates a box. Its `texture` can be `None`, a PNG path, or a NumPy image. The supplied `cube_texture.png` has six square faces arranged as:

$$\begin{array}{ccc} \text{red } +Y & \text{green } -Y & \text{blue } -X \\ \text{yellow } +X & \text{magenta } +Z & \text{cyan } -Z \end{array}$$

`ArucoCube` inherits from `Cube` and generates an ArUco texture. Set `marker_faces=1` for a marker on the local top face, or `marker_faces=6` to repeat it on every face.

Option	Values	Meaning
`physics`	`dynamic`	Responds to forces; movable in object mode.
	`kinematic`	Movable in object mode; unaffected by forces.
	`fixed`	Anchored to the world; unavailable in object mode.
`gravity`	`True`/`False`	Enables or compensates gravity for dynamic objects.
`collision`	`True`/`False`	Makes an object solid or lets bodies pass through it.

The `_CUBES` scenario is a compact comparison of these combinations. It is the best place to experiment with the settings.

5 Changing the Robot Start Pose

```
RobotPose(
    position=(0.5, 0.0, 0.0),
    orientation=(0.7071, 0.0, 0.0, 0.7071),
)
```

Position uses world metres. Orientation is a quaternion in `(w, x, y, z)` order. The default pose is `(0, 0, 0)` with identity orientation `(1, 0, 0, 0)`.

6 RoboCasa Environments

Set `robocasa.enabled=True` to replace the default table scene with a generated RoboCasa kitchen. `task` selects the kitchen activity, while `layout` and `style` choose its geometry and appearance. Imported objects work in RoboCasa exactly as they do in the default scene.

```
SimConfig(
    robocasa=RoboCasaConfig(
        enabled=True,
        task="PnPCounterToCab",
        layout=0,
        style=0,
    ),
    objects=[...],
)
```

Figure 3: A custom object imported into a generated RoboCasa kitchen.

7 Interactive Object Controls

Scenes with dynamic or kinematic objects automatically enable object controls. Press `T` or gamepad START to enter object mode; normal teleop input is temporarily hidden. Press `T` again to return to robot control and print the changed objects' reusable poses.

Keyboard	**Gamepad**	**Action**
`M` / `N`	RB / LB	Select next / previous object
`W` / `S`	Left stick Y	World $+Y$ / $-Y$
`D` / `A`	Left stick X	World $+X$ / $-X$
`Z` / `X`	Right stick Y	World $+Z$ / $-Z$
`U/O`, `I/K`, `J/L`	LT + sticks	Roll, pitch, yaw
`G` / `F`	Y / B	Disable / enable gravity

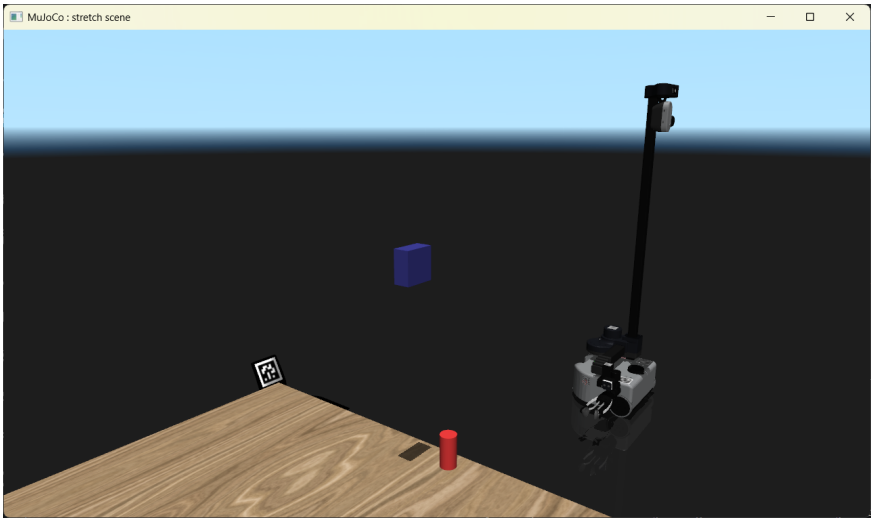

Figure 4: An object being repositioned in object-control mode.

8 Creating Your Own Scene

Copy an included `SimConfig`, rename it, and select it with `CONFIG`. Give each object a unique `name`, then set its `position`, `size` or `scale`, and the physics options that differ from their defaults. Use object controls to place objects visually; leaving object mode prints a pose that can be copied into `sim_config.py` for the next run.